

Thực hành – Tuần 1

Máy trừu tượng



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 3 tháng 3 năm 2026

Mục lục

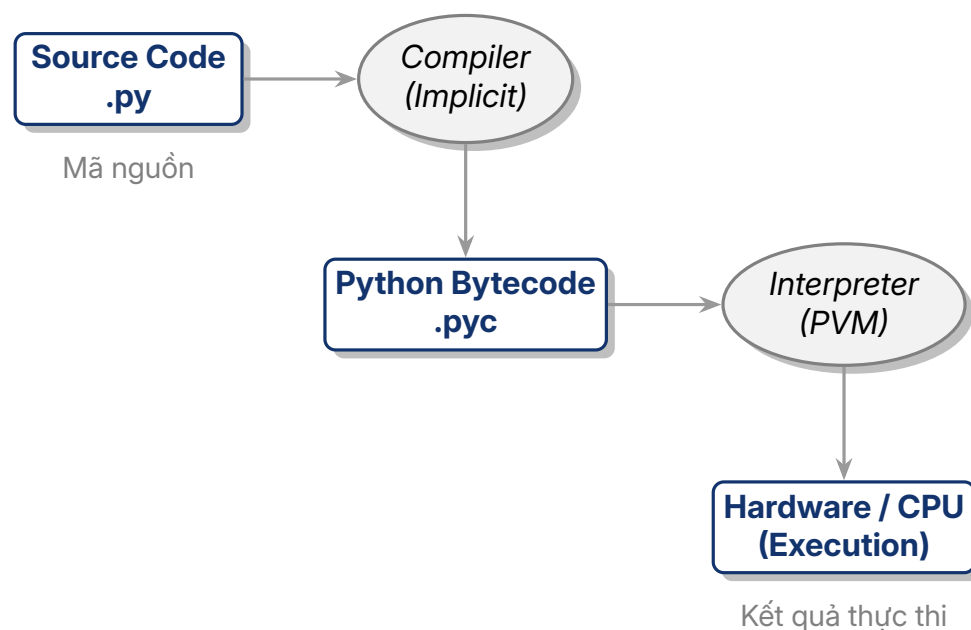
I	Nội dung thực hành	2
I.1	Kiến thức	2
I.2	Mục tiêu	3
II	Cài đặt môi trường	4
II.1	Cài đặt trình biên dịch G++	4
II.1.1	Hệ điều hành Windows	4
II.1.2	Hệ điều hành Ubuntu/Linux	5
II.1.3	Hệ điều hành MacOS	6
II.2	Thiết lập môi trường VSCode	7
II.3	Setup Server chấm bài Coding	11
III	Bài tập thực hành	14
IV	Bài tập về nhà	15

I. Nội dung thực hành

I.1 Kiến thức

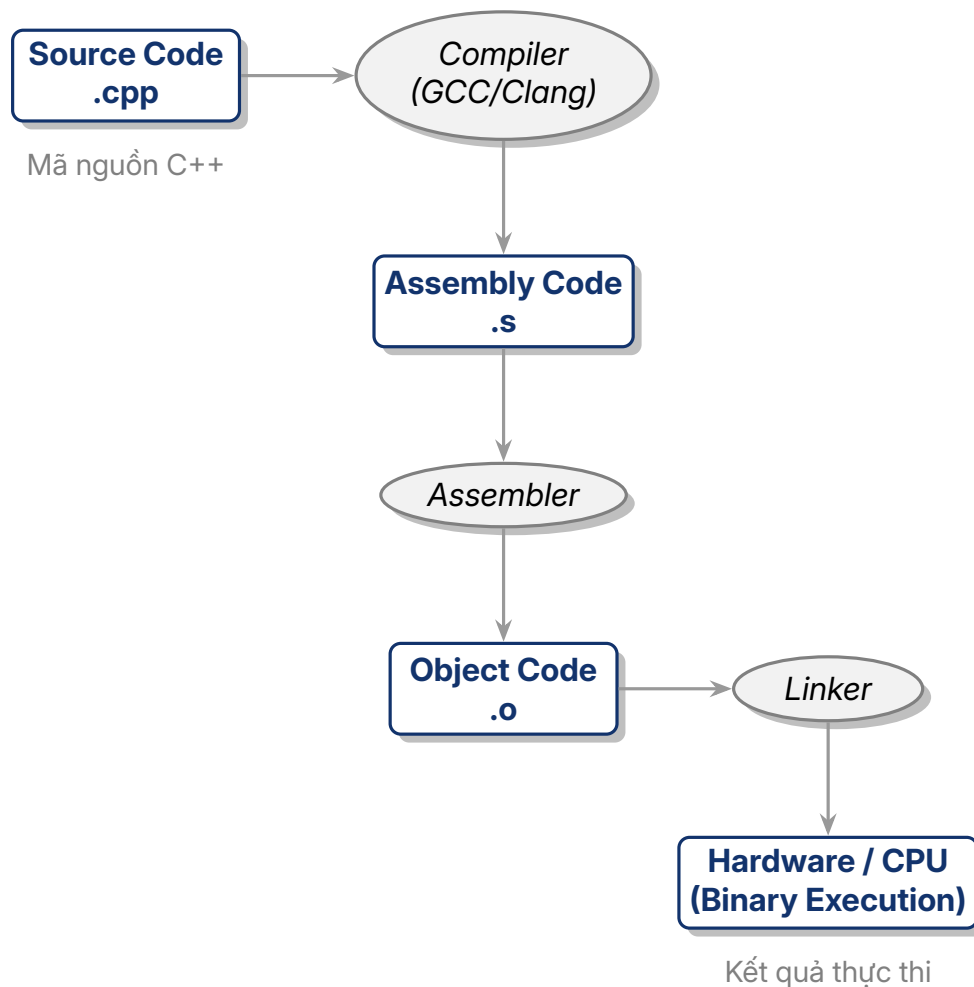
Qua buổi học lý thuyết, các bạn đã được làm quen với các khái niệm *Máy trừu tượng* (*Abstract Machine*) $\mathcal{M}_{\mathcal{L}}$ của một *ngôn ngữ lập trình* $\mathcal{L}$; *Bộ thông dịch* (*Interpreter*), cũng như hai phương thức thực thi chính: *Thông dịch* (*Interpretation*) và *Biên dịch* (*Compilation*). Trong phần này, để hiểu rõ hơn các khái niệm này ở một mức vận dụng sâu hơn thông hiểu; chúng ta sẽ tiếp cận với ngôn ngữ lập trình C++.

Thực tế, các bạn đã tiếp xúc với quá trình *Thông dịch* thông qua ngôn ngữ Python trong môn **Tư duy tính toán**. Tuy nhiên, cần lưu ý rằng Python không hoàn toàn là một ngôn ngữ thông dịch thuần túy (*purely interpreted implementation*). Nếu đọc kỹ hơn trong giáo trình, mã nguồn Python được biên dịch thành mã trung gian gọi là Python bytecode. Sau đó, một thực thể phần mềm là *Máy ảo Python* *Python Virtual Machine* - *PVM* sẽ đóng vai trò "*máy chủ thực thi*" (*Host Machine*) để thông dịch mã này.



Hình 1: Quá trình thông dịch mã nguồn Python

Ngược lại, C++ được *Biên dịch* trực tiếp sang mã máy (Native Code) và khác với Python, đây là một ngôn ngữ biên dịch thuần túy (*purely compiled implementation*). Cụ thể, mã nguồn C++ chuyển hóa qua tầng *hợp ngữ* (Assembly), sau đó thành mã đối tượng (Object Code) và cuối cùng được *Trình liên kết* (Linker) đóng gói thành tệp thực thi (đuôi .exe). Điểm khác biệt cốt lõi nằm ở "*Máy chủ thực thi*" (*Host Machine*): đối với C++, đó là CPU thuộc tầng *phần cứng* (*Hardware*); còn với python, đó là PVM thuộc tầng *phần mềm* (*Software*).



Hình 2: Quá trình biên dịch mã nguồn C++

Sự khác biệt về kiến trúc này dẫn đến một sự đánh đổi (*trade-off*) điển hình trong khoa học máy tính:

- Về **hiệu năng**: C++ chiếm ưu thế tuyệt đối do mã lệnh được tối ưu hóa riêng cho phần cứng, thực thi trực tiếp mà không cần lớp trung gian.
- Về **sự linh hoạt và tính di động**: Python vượt trội nhờ khả năng "viết một lần, chạy mọi nơi". PVM đóng vai trò lớp đệm giúp che giấu sự phức tạp của phần cứng bên dưới, trong khi file thực thi của C++ thường phụ thuộc chặt chẽ vào hệ điều hành và kiến trúc CPU cụ thể.

I.2 Mục tiêu

Trong bài tập tuần này, các bạn sẽ được làm việc với trình biên dịch G++ trong bộ công cụ *GNU Compiler Collection (GCC)*. Dưới đây là những mục tiêu của buổi học:

1. Cài đặt trình biên dịch G++, tương ứng từng hệ điều hành:

- Hệ điều hành **Windows**: Cài đặt phần mềm môi trường MSYS2, sử dụng trình quản lý gói pacman để cài đặt các gói thư viện C++ và trình biên dịch G++;

- Hệ điều hành **Ubuntu/Linux**: Sử dụng công cụ quản lý gói apt để cài đặt các gói thư viện C++ và trình biên dịch G++;
 - Hệ điều hành **MacOS**: Cài đặt tiện ích dòng lệnh xcode-select, thông qua cài đặt thì trình biên dịch G++ (wrapper tới Clang++) sẽ được cài đặt.
2. Thiết lập biên dịch mã nguồn C++ trong VSCode (bao gồm quá trình biên dịch và thực thi).
 3. Thiết lập server thực thi mã nguồn C++, để thực hành bài tập lập trình trên trang *UET Coding*.
 4. Thực hành lập trình ngôn ngữ C++ sử dụng VSCode.
 5. Thực hành lập trình ngôn ngữ C++ trên trang *UET Coding*

II. Cài đặt môi trường

II.1 Cài đặt trình biên dịch G++

II.1.1 Hệ điều hành Windows

Trong bài học này, chúng ta sẽ sử dụng môi trường MSYS2 UCRT64.

II.1.1.1. Cài đặt môi trường MSYS2

Bước 1: Tải bộ cài đặt

Truy cập trang chủ của MSYS2 tại: <https://www.msys2.org> và tải phiên bản mới nhất dành cho Windows.

Bước 2: Cài đặt

- Chạy file cài đặt vừa tải về.
- Chọn thư mục cài đặt mặc định (khuyến nghị: C:\msys64).
- Hoàn tất quá trình cài đặt.

Bước 3: Cập nhật hệ thống

Sau khi cài đặt xong, mở MSYS2 MSYS và thực hiện lệnh sau để cập nhật toàn bộ hệ thống:

```
1 pacman -Syu
```

Nếu cửa sổ tự động đóng sau khi cập nhật, hãy mở lại MSYS2 và tiếp tục chạy lại lệnh trên cho đến khi hệ thống được cập nhật hoàn toàn.

II.1.1.2. Cài đặt trình biên dịch G++

Bước 1: Mở môi trường UCRT64

Từ menu Start của Window, tìm kiếm và chọn MSYS2 UCRT x64. Cửa sổ **bash shell** của MSYS2 UCRT64 hiện ra.

Bước 2: Cài đặt trình biên dịch G++

Trong cửa sổ UCRT64, chạy lệnh sau:

```
1 pacman -S mingw-w64-ucrt-x86_64-gcc
```

Khi được hỏi xác nhận cài đặt, nhập Y và nhấn Enter.

Bước 3: Kiểm tra cài đặt

Sau khi cài đặt xong, kiểm tra phiên bản G++ bằng lệnh:

```
1 g++ --version
```

Nếu màn hình hiển thị thông tin phiên bản như dưới đây, quá trình cài đặt đã thành công:

```
1 g++.exe (Rev11, Built by MSYS2 project) 15.2.0
2 Copyright (C) 2025 Free Software Foundation, Inc.
3 This is free software; see the source for copying conditions.  There
  is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
  PARTICULAR PURPOSE.
```

II.1.2 Hệ điều hành Ubuntu/Linux

Trên các bản phân phối dựa trên Debian như Ubuntu, việc cài đặt trình biên dịch rất đơn giản thông qua kho lưu trữ chính thức của hệ thống.

II.1.2.1. Cài đặt trình biên dịch G++

Bước 1: Cập nhật danh sách gói

Mở Terminal (Ctrl+Alt+T) và thực hiện cập nhật danh sách các gói phần mềm để đảm bảo bạn cài đặt phiên bản mới nhất:

```
1 sudo apt update
```

Bước 2: Cài đặt gói công cụ lập trình

Bạn nên cài đặt gói `build-essential`, gói này bao gồm trình biên dịch `g++`, `gcc` và các công cụ cần thiết khác như `make`:

```
1 sudo apt install build-essential
```

Khi được hỏi xác nhận cài đặt, hãy nhập `Y` (hoặc `S` tùy ngôn ngữ hệ thống) và nhấn `Enter`.

Bước 3: Kiểm tra cài đặt

Sau khi quá trình hoàn tất, kiểm tra sự hiện diện của trình biên dịch bằng lệnh:

```
1 g++ --version
```

Nếu thành công, bạn sẽ nhận được thông báo tương tự như sau (phiên bản có thể thay đổi tùy theo đời Ubuntu):

```
1 g++ (Ubuntu 13.2.0-23ubuntu4) 13.2.0
2 Copyright (C) 2023 Free Software Foundation, Inc.
3 This is free software; see the source for copying conditions.  There
  is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
  PARTICULAR PURPOSE.
```

II.1.3 Hệ điều hành MacOS

Trên MacOS, thay vì cài đặt các bộ công cụ rời rạc, Apple cung cấp gói `Command Line Tools` tích hợp sẵn trình biên dịch `Clang` (nhưng vẫn hỗ trợ câu lệnh `g++` như một bí danh - alias).

II.1.3.1. Cài đặt trình biên dịch qua Xcode Tools

Bước 1: Mở Terminal

Nhấn `Command + Space`, tìm kiếm `Terminal` và nhấn `Enter`.

Bước 2: Cài đặt công cụ dòng lệnh

Nhập lệnh sau để kích hoạt trình cài đặt chính thức từ Apple:

```
1 xcode-select --install
```

Một cửa sổ xác nhận sẽ hiện ra, hãy chọn **Install** (Cài đặt) và chấp nhận các điều khoản. Hệ thống sẽ tự động tải về và cấu hình cho bạn.

Bước 3: Kiểm tra cài đặt

Mặc dù chúng ta sử dụng lệnh `g++`, thực tế MacOS sẽ gọi trình biên dịch Clang. Hãy kiểm tra bằng lệnh:

```
1 g++ --version
```

Kết quả sẽ hiển thị tương tự như sau:

```
1 Apple clang version 15.0.0 (clang-1500.3.9.4)
2 Target: arm64-apple-darwin23.4.0
3 Thread model: posix
```

II.1.3.2. Clang và G++: Sự khác biệt

Dù đều là lệnh `g++` trên Linux hay MacOS, quy trình cốt lõi là **không đổi**:

Source Code → Compile → Machine Code → Linker → Executable

Tuy nhiên, có một số điểm khác biệt như sau:

- **Gốc gác:** G++ thuộc dự án **GNU**, vốn là tiêu chuẩn lâu đời trên Linux. Trong khi đó, Clang dựa trên nền tảng **LLVM**, được Apple và các tập đoàn lớn phát triển để tối ưu hóa hiệu năng và tốc độ biên dịch.
- **Thư viện tiêu chuẩn:** G++ sử dụng `libstdc++`, còn Clang trên MacOS thường sử dụng `libc++`. Điều này có thể dẫn đến sự khác biệt nhỏ về cách xử lý bộ nhớ hoặc các tính năng C++ mới nhất.
- **Tính tương thích:** Clang được thiết kế để tương thích ngược với các câu lệnh của GCC, đó là lý do tại sao ta vẫn có thể dùng lệnh `g++` trên MacOS để biên dịch mà không gặp lỗi cú pháp.

Tóm lại: Trong môn học này, dù khác nhau về hạ tầng (LLVM vs GNU), nhưng sự khác biệt này có thể chưa đủ lớn để tác động tới quá trình học tập và thực thi các mã nguồn C++ trên cả hai hệ điều hành. Do đó, ở thời điểm hiện tại, trình biên dịch Clang vẫn được chấp nhận sử dụng.

II.2 Thiết lập môi trường VSCode

Bước 1: Cài đặt Visual Studio Code

Tải và cài đặt VSCode tại trang chủ: <https://code.visualstudio.com>. Bước này có thể bỏ qua nếu đã cài đặt VSCode ở môn học **Tư duy tính toán**.

Bước 2: Cài đặt Extension cần thiết

Mở VSCode và cài đặt các extension sau:

- **C/C++** (khuyến nghị)
- **C/C++ DevTools** (tùy chọn)
- **C/C++ Themes** (tùy chọn)
- **CMake Tools** (tùy chọn, trong các bài học tương lai có thể cần thiết)

Hoặc, cài đặt bộ extension **C/C++ Extension Pack** bao gồm toàn bộ 4 extension trên.

Bước 3: Thêm compiler vào biến môi trường PATH

Để VSCode có thể gọi trình biên dịch ở bất cứ đâu, chúng ta cần cấu hình biến môi trường hệ thống:

- **Windows:** Thêm đường dẫn sau:

```
1 C:\msys64\ucrt64\bin
```

vào biến môi trường PATH (thực hiện qua *System Properties*).

- **Ubuntu/Linux & MacOS:** Có thể bỏ qua bước này. Thông thường, sau khi cài đặt qua apt hoặc xcode-select, trình biên dịch đã tự động nằm trong đường dẫn chuẩn (/usr/bin).

Lưu ý: Sau khi cấu hình, hãy khởi động lại VSCode để thay đổi có hiệu lực.

Bước 4: Viết mã nguồn chương trình C++

Tạo folder programming-methodology (khuyến nghị đặt tên tiếng anh, không có dấu và dấu cách). Với mỗi tuần học, nên tạo một folder riêng (ví dụ week-1). Trong folder đó, tạo file main.cpp với nội dung ví dụ:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello, World!" << endl;
6     return 0;
7 }
```

Bước 5.1: Biên dịch và chạy chương trình C++ (thủ công)

Sau khi đã có trình biên dịch và mã nguồn. Dưới đây là hướng dẫn cách làm **thủ công**. Mở Terminal tích hợp trong VSCode (Ctrl + `) và thực hiện:

```
1 g++ main.cpp -o main
```


Đoạn lệnh này sẽ *biên dịch* mã nguồn .cpp sang mã object .o và tạo ra file thực thi main (một số trường hợp sẽ là main.exe). Để thực thi (chạy) chương trình C++ vừa được biên dịch, chạy lệnh sau trong cửa sổ Terminal:

- **Windows:**

```
1 ./main.exe
```

- **Linux & MacOS:**

```
1 ./main
```

Nếu chương trình in ra dòng sau:

```
1 Hello, World!
```

Bạn đã thiết lập thành công môi trường lập trình C++ với VSCode, biên dịch và thực thi một chương trình C++ **bằng cách thủ công**.

Bước 5.2: Biên dịch và chạy chương trình C++ (tự động)

Vì mỗi lần biên dịch mã nguồn phải thực hiện thủ công như trên sẽ mất nhiều thời gian. Do đó, trong VSCode có hỗ trợ thiết lập tiến trình **Run Build Task** để rút ngắn thời gian cho lập trình viên.

Cách thiết lập như sau; trong cửa sổ folder môn học, tạo một folder mới với tên .vscode (một tên folder đặc biệt để VSCode tự động nhận diện các tùy chỉnh riêng từ người dùng). Trong folder này, tạo một file với tên tasks.json và điền nội dung sau:

```
1 {
2     "version": "2.0.0",
3     "tasks": [
4         {
5             "type": "shell",
6             "label": "Build C++ (MSYS2 g++ C++20)",
7             "command": "C:\\msys64\\ucrt64\\bin\\g++.exe",
8             "args": [
9                 "-std=c++20",
10                "-fdiagnostics-color=always",
11                "-Wall",
12                "-Wextra",
13                "-g",
14                "${file}",
15                "-o",
16                "${fileDirname}\\${fileBasenameNoExtension}.exe"
17            ],
18            "options": {
```

```

19         "cwd": "${fileDirname}"
20     },
21     "problemMatcher": [
22         "$gcc"
23     ],
24     "group": {
25         "kind": "build",
26         "isDefault": false
27     },
28     "detail": "MSYS2 UCRT64 g++ 15.x (C++20)"
29 }
30 ]
31 }

```

Đây là cấu hình một **Task** với các thông tin về nơi chứa trình biên dịch, các tham số biên dịch và thiết lập thông tin cơ bản. Cụ thể hơn như sau:

- **type**: loại shell cho biết đây là một tiến trình chạy lệnh trực tiếp trên terminal.
 - **label, detail**: tên hiển thị và mô tả chi tiết để bạn chọn đúng Task khi có nhiều cấu hình.
 - **command**: đường dẫn tuyệt đối dẫn đến trình biên dịch g++ .exe của MSYS2.
 - *Lưu ý cho Linux/MacOS*: Thay bằng đường dẫn ngắn gọn là "g++" vì hệ thống đã tự nhận diện được trình biên dịch trong PATH.
 - **args**: danh sách các tham số truyền vào trình biên dịch, bao gồm:
 - -std=c++20: Yêu cầu trình biên dịch tuân thủ tiêu chuẩn C++20 mới nhất.
 - -fdiagnostics-color=always: Luôn hiển thị màu sắc trong thông báo lỗi để dễ quan sát.
 - -Wall: Bật tất cả các cảnh báo (warnings) phổ biến về lỗi logic trong mã nguồn.
 - -Wextra: Bật thêm các cảnh báo nâng cao để đảm bảo mã nguồn tối ưu và an toàn.
 - -g: Tạo thông tin gỡ lỗi (debug), cho phép bạn sử dụng công cụ Debugger sau này.
 - \${file}: Biến tự động của VSCode, đại diện cho file mã nguồn đang mở.
 - -o: Tham số chỉ định tên file đầu ra sau khi biên dịch thành công.
 - \${fileDirname}\\\${fileBasenameNoExtension}.exe: Đường dẫn file thực thi kết quả (cùng thư mục và cùng tên với file mã nguồn).
 - *Lưu ý đa hệ điều hành*: Trên **Linux/MacOS**, cần bỏ đuôi .exe và sử dụng dấu xet xuôi (/) thay vì xet ngược (\\).
- Ví dụ: \${fileDirname}/\${fileBasenameNoExtension}.

Có thể thấy, việc biên dịch một mã nguồn C++ có một khuôn mẫu nhất định: đều sử dụng một trình biên dịch, với các tham số tương đồng và chỉ có khác biệt nằm ở tên file mã nguồn cũng như đầu ra.

```
1  "-g",
2  "${file}",
3  "-o",
4  "${fileDirname}\\${fileBasenameNoExtension}.exe"
```

Để biên dịch, giờ chỉ cần mở mã nguồn C++ và bấm tổ hợp phím **Ctrl + Shift + B** (hoặc **Command + Shift + B** trên MacOS). Một cửa sổ nhỏ ở giữa bên trên cùng VSCode sẽ hiện ra, yêu cầu lựa chọn tiến trình thực hiện. Chọn tiến trình **Build C++ (MSYS2 g++ C++20)** (hoặc tên theo **label** được cấu hình). VSCode cửa sổ phía dưới sẽ tự động chuyển sang **OUTPUT** và hiển thị kết quả biên dịch chương trình.

Để thực thi, chạy lệnh sau trong cửa sổ Terminal:

- **Windows:**

```
1 ./main.exe
```

- **Linux & MacOS:**

```
1 ./main
```

II.3 Setup Server chấm bài Coding

Bước 1: Tải file Server thực thi

Tải file `judger_dev.exe` từ đường dẫn do giảng viên cung cấp tại: [Google Drive \(Server Download\)](#). Chọn phiên bản phù hợp với hệ điều hành đang sử dụng:

- **Windows:** Tải `windows-v0.1.zip`, giải nén được file `judger_dev.exe`.
- **Linux/MacOS:** Tải phiên bản tương ứng hệ điều hành (ví dụ `linux-v0.1.zip` hoặc `macos-v0.1.zip`), giải nén được file `judger_dev.bin`.

Để chạy file này, sinh viên **không cần cài đặt thêm bất kỳ thư viện Python nào**. (Chỉ trong một số trường hợp đặc biệt mới cần bổ sung file hệ thống như `.dll` nếu Windows yêu cầu.)

Sau khi tải về, tạo một folder riêng (ví dụ: `coding-server`) và đặt file:

```
1 judger_dev.exe
```

vào trong folder này.

Bước 2: Chuẩn bị file cấu hình `.env`

Trong cùng thư mục với `judger_dev.exe`, tạo một file mới có tên:

```
1 .env
```

File `.env` có thể có sẵn sau khi giải nén file `.zip`. Mở file này và điền đường dẫn tuyệt đối đến các chương trình cần thiết:

```
1 # Windows
2 GPP_BIN=C:\msys64\ucrt64\bin\g++.exe
3 PYTHON_BIN=C:\Users\YourName\AppData\Local\Programs\Python\Python313\
  python.exe
4
5 # Linux/MacOS
6 GPP_BIN=/usr/bin/g++
7 PYTHON_BIN=/usr/bin/python3
```

Giải thích:

- `GPP_BIN`: là đường dẫn tới chương trình `g++` (đã được hướng dẫn cài đặt ở phần trước).
- `PYTHON_BIN`: là đường dẫn tới chương trình `python` (đã được cài đặt trong môn **Tư duy tính toán**, sinh viên có thể sử dụng lại). Trên Linux/MacOS thường là `/usr/bin/python3`.

Lưu ý:

- Đường dẫn phải là đường dẫn tuyệt đối (absolute path).
- Không thêm khoảng trắng thừa.
- Không đặt dấu ngoặc kép.

Bước 3: Khởi động Server chấm bài

Mở Terminal tại thư mục chứa `judger_dev.exe` và chạy:

- **Windows:**

```
1 judger_dev.exe
```

- **Linux & MacOS:**

```
1 judger_dev.bin
```

Nếu cấu hình đúng, màn hình sẽ hiển thị thông báo server đang chạy tại:

```
1 http://127.0.0.1:8001
```

Quan trọng: Hãy đọc kỹ phần log khi khởi động để kiểm tra:

- Đường dẫn env_path
- Giá trị GPP_BIN
- Giá trị PYTHON_BIN

Nếu một trong các đường dẫn sai, server sẽ không thể biên dịch hoặc thực thi chương trình.

Bước 4: Truy cập và sử dụng

Mở trình duyệt (Chrome, Edge, Firefox, ...) và truy cập địa chỉ:

```
1 https://coding.uet.edu.vn/
```

Làm như hướng dẫn của môn **Tư duy tính toán**: chạy server trước, sau đó truy cập vào trang Coding; kiểm tra server đã kết nối với trang web chưa.

Để biên dịch và thực thi bài làm bằng g++, trong mã nguồn làm bài sẽ cần để ở dòng đầu tiên đoạn mã `// .cpp`, ví dụ như:

```
1 // .cpp
2 #include <iostream>
3 using namespace std;
4 ...
```

Để thực thi bài làm bằng python, trong mã nguồn làm bài cần để ở dòng đầu tiên đoạn mã `#.py`, ví dụ như:

```
1 #.py
2 import sys
3 if __name__ == "__main__":
4 ...
```

Sau mỗi lần chấm bài, một thư mục `.cache` có thể được tạo để lưu trữ tạm thời các file trung gian.

Một số lỗi thường gặp

- **Permission Denied (Linux/MacOS)**: Chạy lệnh `chmod +x judge_dev.bin` để cấp quyền thực thi cho file.
- **Không tìm thấy g++.exe hoặc không biên dịch được mã nguồn C++**: kiểm tra lại biến `GPP_BIN` trong file `.env`.
- **Không tìm thấy python.exe hoặc không biên dịch được mã nguồn python**: kiểm tra lại biến `PYTHON_BIN`.
- **Port 8001 đã được sử dụng**: đóng các tiến trình server cũ hoặc khởi động lại máy.

Lưu ý quan trọng

- Sinh viên **chỉ sử dụng file thực thi** (.exe hoặc .bin) được cung cấp chính thức.
- Không chỉnh sửa, dịch ngược hoặc can thiệp vào logic chấm bài.
- Không thay đổi nội dung thư mục .cache trong quá trình server đang chạy.

III. Bài tập thực hành

Bài tập 1: Xuất thông tin sinh viên

Mô tả:

Viết chương trình C++ để in thông tin cá nhân cơ bản ra màn hình console.

Yêu cầu kỹ thuật:

- Sử dụng trình biên dịch g++ phiên bản 15.x trở lên.
- Thực hiện biên dịch thủ công qua Terminal và tự động qua VSCode.
- Sử dụng `std::cout` và `std::endl`.

Input:

Không có dữ liệu đầu vào.

Output:

```
Nguyen Van A  
23020001
```

Lưu ý:

Hãy chú ý cấu trúc thư mục làm việc (week-1/ex1.cpp) để tasks.json hoạt động chính xác.

IV. Bài tập về nhà

Làm các bài tập chủ đề **Máy trừu tượng** (*[Abstract Machine]*) trên trang Coding. Liên hệ với giảng viên thực hành về thời hạn làm bài.